CS543 · Web Services · Node & Express

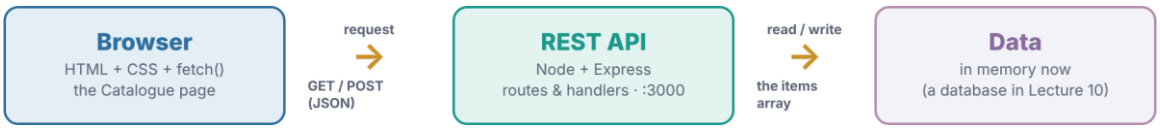
Node & Express Tutorial

Build one small service end to end — HTML/CSS → REST API (Node + Express) → data

> **What you will build**
> A "Catalogue" page that lists dishes and a form to add one — the page talks to a REST API (Node + Express) that holds the menu. One **read** (GET) and one **write** (POST).

The architecture

Three layers. The browser never touches the data directly — it only sends HTTP requests to the API, and the API (your Express routes) reads and writes the data.



Browser (front-end) → REST API (Node + Express) → data (in memory now; a database in Lecture 10)

> **The task, in one line**
> Serve a menu of dishes over REST; let a web page ask "**list every dish**" and "**add a new dish**" with a form.

Part 0 · Install the tools

Install these first. On any OS (Windows / macOS / Linux).

Node.js (v18 or newer) — runs your server, and brings `npm` with it. Download from `nodejs.org` and install.

```
Terminal
$ node -v
v22.x.x
$ npm -v
10.x.x
```

- **VS Code** (code editor) — any editor works, but VS Code is convenient for this course.

Part 1 · Set up the project

Make a project folder, e.g. `catalogue`, step into it, create the project, and install Express:

```
Terminal
$ mkdir catalogue && cd catalogue
$ npm init -y                # creates package.json
$ npm install express        # the web framework
```

One package — `express` — is all we need. **npm** records it in `package.json` and downloads it into `node_modules/` (which you never commit to git).

Part 2 · Your first server

Create `server.js`. This is the smallest possible Express server — it answers one request, `GET /`, with a line of text:

```
server.js
// server.js
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Catalogue is alive');
});
```

```
app.listen(3000, () => console.log('on http://localhost:3000'));
```

Run it, and open the address in your browser:

```
Terminal
$ node server.js
on http://localhost:3000
```

Checkpoint

Open `http://localhost:3000` — you should see **Catalogue is alive**. That reply answered the browser's `GET /` request. Stop the server any time with `Ctrl + C`.

Part 3 · Serve the menu (the read routes)

Now give the server some data and two **read** routes. Replace `server.js` with the version below — it adds an in-memory `items` array, `GET /items` (the whole menu) and `GET /items/:id` (one dish, or a 404).

```
server.js
// server.js
const express = require('express');
const app = express();
app.use(express.json());            // read JSON request bodies (Part 4)
app.use(express.static('public'));  // serve the web page (Part 5)

// the menu -- in memory for now (a real database in Lecture 10)
let items = [
  { id: 1, name: 'Veg Thali',   price: 90 },
  { id: 2, name: 'Paneer Roll', price: 70 },
];
let nextId = 3;

// GET /items  -- list the whole menu
app.get('/items', (req, res) => {
  res.json(items);
});

// GET /items/:id  -- one dish, or 404
app.get('/items/:id', (req, res) => {
  const item = items.find(i => i.id === Number(req.params.id));
  if (!item) return res.status(404).json({ error: 'not found' });
  res.json(item);
});

app.listen(3000, () => console.log('Catalogue on http://localhost:3000'));
```

Restart (`Ctrl + C`, then `node server.js`) and test the two routes from a second terminal with `curl` — or just open the URLs in a browser:

```
Terminal
$ curl localhost:3000/items
[{"id":1,"name":"Veg Thali","price":90},{"id":2,"name":"Paneer Roll","price":70}]

$ curl localhost:3000/items/1
{"id":1,"name":"Veg Thali","price":90}

$ curl -i localhost:3000/items/99
HTTP/1.1 404 Not Found
{"error":"not found"}
```

Why 404, not an empty 200?

When the dish doesn't exist, the status code **is** the answer — `404 Not Found`, not a 200 with an empty body. The client can tell success from failure by the code alone (Lecture 6).

Part 4 · Add a dish (the write route)

Add a **create** route. The client POSTs a JSON body; `express.json()` (already added in Part 3) fills `req.body` with it. Add this route above `app.listen`:

server.js (add this route)

```
// POST /items  -- add a dish
app.post('/items', (req, res) => {
  const { name, price } = req.body;          // what the client sent
  if (!name) return res.status(422).json({ error: 'name required' });
  const item = { id: nextId++, name, price: Number(price) || 0 };
  items.push(item);
  res.status(201).json(item);                // 201 Created + the new dish
});
```

Restart, then create a dish with `curl`. The `-X POST` sets the method, and `-d` sends the JSON body:

Terminal

```
$ curl -i -X POST localhost:3000/items \
    -H "Content-Type: application/json" \
    -d '{"name":"Masala Dosa","price":60}'

HTTP/1.1 201 Created
{"id":3,"name":"Masala Dosa","price":60}
```

The request → response pattern

The client **sent** a method + path + JSON body (`POST /items`); your route **validated** it, created the dish, and **replied** with `201` and the new item. Every REST endpoint is this shape.

Part 5 · Build the front-end (HTML + CSS)

The server already serves a `public` folder (Part 3's `express.static`). Create `public/style.css`:

public/style.css

```
* { box-sizing: border-box; }
body { font-family: system-ui, Arial, sans-serif; background: #f1f3f8; color: #1b1b2f; margin: 0; padding:
2rem; }
h1 { color: #2e6f9e; margin: 0 0 0.3rem; }
.sub { color: #5d6577; margin: 0 0 1.5rem; }
.dish {
  display: flex; justify-content: space-between; align-items: center;
  background: #fff; border: 1px solid #d6dae6; border-left: 6px solid #1b998b;
  border-radius: 10px; padding: 0.8rem 1.1rem; margin-bottom: 0.7rem;
  box-shadow: 0 2px 8px rgb(0 0 0 / 6%);
}
.dish .name { font-weight: 600; }
.dish .price { color: #2e6f9e; font-weight: 700; }
form { background: #fff; border: 1px solid #d6dae6; border-left: 6px solid #e8541e; border-radius: 10px;
padding: 1rem 1.25rem; margin-top: 1.5rem; }
form h3 { margin: 0 0 0.7rem; color: #e8541e; }
input { padding: 0.5rem; margin: 0.25rem 0.4rem 0.25rem 0; border: 1px solid #c9ccd6; border-radius: 6px; }
button { padding: 0.55rem 1rem; background: #2e6f9e; color: #fff; border: 0; border-radius: 6px; cursor:
pointer; font-weight: 600; }
```

Now `public/index.html`. The `<script>` at the bottom does two things: it `fetches GET /items` to draw the list, and the form sends a `POST /items` to add a dish, then refreshes. Because Express serves this page, `fetch('/items')` talks to the same server — no CORS to worry about.

public/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CampusEats Catalogue</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>CampusEats Catalogue</h1>
  <p class="sub">The menu &amp;mdash; served by a REST API (Node + Express)</p>

  <div id="list">Loading&amp;hellip;</div>
```

```
  <form id="addForm">
    <h3>Add a dish</h3>
    <input id="name" placeholder="Dish name" required>
    <input id="price" type="number" placeholder="Price (rupees)">
    <button type="submit">Add dish</button>
  </form>

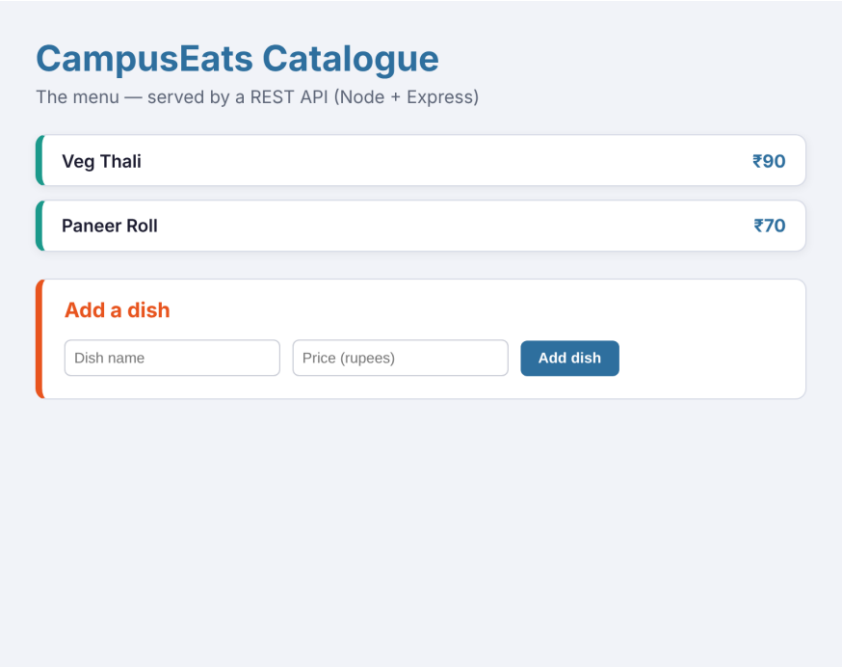
  <script>
    // 1. READ: GET /items, then draw the list
    async function load() {
      const res = await fetch('/items');
      const items = await res.json();
      document.getElementById('list').innerHTML = items.map(i =>
        `<div class="dish"><span class="name">${i.name}</span>` +
        `<span class="price">&#8377;${i.price}</span></div>`).join('');
    }

    // 2. WRITE: the form POSTs a new dish, then refreshes the list
    document.getElementById('addForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const name  = document.getElementById('name').value;
      const price = document.getElementById('price').value;
      await fetch('/items', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name, price })
      });
      e.target.reset();
      load();
    });

    load();
  </script>
</body>
</html>
```

Part 6 · Run the whole app

Start the server (`node server.js`) and open `http://localhost:3000` in your browser. The page loads and immediately fetches the menu from your API:

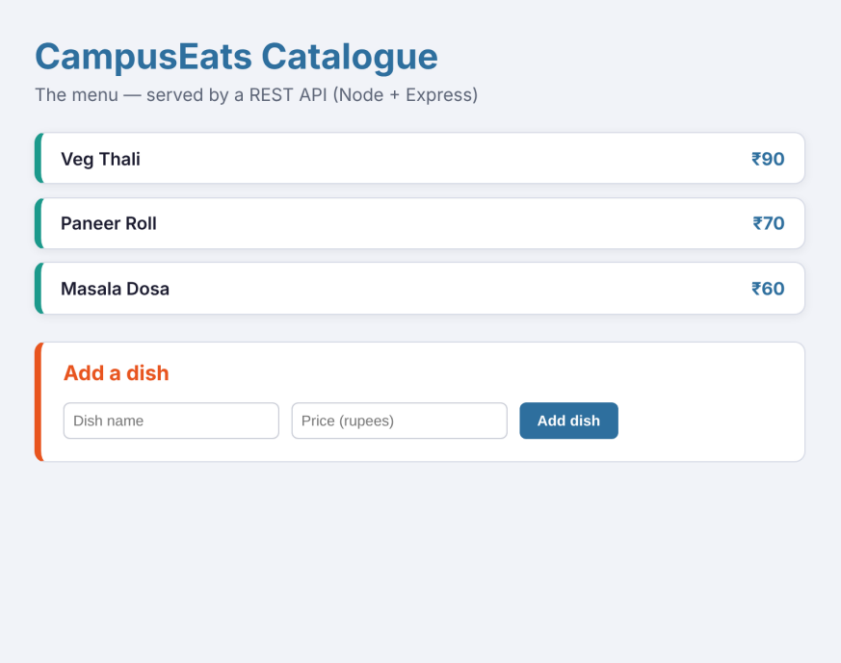


The live Catalogue page — the two dishes are fetched from GET /items the moment the page loads.

If the list says "Loading…" forever
The browser couldn't reach the API. Make sure `node server.js` is running and printing port 3000, and that you opened `http://localhost:3000` (not the file directly). Check the browser Console (F12) for errors.

Part 7 · Add a dish (the form)

In the **Add a dish** form, type a name (e.g. `Masala Dosa`), a price (`60`), and click **Add dish**. The form sends a `POST /items`, the server adds it to the array, and the list refreshes — the new dish appears instantly:



After the POST — "Masala Dosa ₹60" now shows in the list, served straight back by GET /items.

> **Remember: in memory = forgetful**
> Stop and restart the server and "Masala Dosa" is gone — the array resets. That is fine for learning the API shape. **Lecture 10** swaps the array for a real database so the data survives restarts.

Recap — what you just did

- Created a Node project and installed **Express** with npm.
- Wrote a **REST API**: `GET /items`, `GET /items/:id` (with a 404), and `POST /items` (with a 201).
- Used `req.params` (the id from the path) and `req.body` (the JSON the client sent).
- Built a plain HTML/CSS page that `fetch`es the list and POSTs a new dish — the whole request → response cycle, live.

Your final project folder:

```
project structure
catalogue/
  server.js           <- the Express REST API
  package.json
  node_modules/       <- installed packages (not committed)
  public/
    index.html        <- the web page (fetch + form)
    style.css
```

> **Where to go next**
> Try adding a `DELETE /items/:id` route (find it, remove it, reply `204`), or a `GET /items?veg=true` filter using `req.query`. Each is a few lines — the same pattern you just followed. Then Lecture 10 gives the menu a real database.